4.8.1 Storing timeseries data

I chose Amazon DynamoDB as the primary time series data store for my home digital twin. While not purpose-built for time series like InfluxDB or TimescaleDB, DynamoDB's serverless architecture eliminates infrastructure management and provides pay-per-use pricing—my most important requirement—while providing single-digit millisecond performance at any scale. DynamoDB's key-value design has the one crucial requirement that you know your access patterns before storing data. This query-first approach demands more upfront planning but delivers highly optimized performance for actual use cases rather than theoretical flexibility. For home digital twins with well-defined patterns and modest data volumes, this trade-off favors DynamoDB's operational simplicity over specialized time series features.

I use a single table design with a simple schema reflecting the sensor data it stores. The partition key is a unique sensor identifier (device EUI for LoRaWAN sensors, descriptive key for manual uploads), the sort key is the measurement timestamp, and attributes contain the relevant readings for each sensor type. This structure accommodates all my home sensors and enables time-window queries, as shown in table 4.3.

Table 4.3 A single DynamoDB table stores all the timeseries data produced by sensors in my home and allows efficient retrieval.

Partition key	Sort key	Humidity	Temperature	CO2	Power	Current	pH
a84041ce41845d13	1749945584458	57.9	18.4	-	-	-	-
a840411971871c86	1749946784664	66.5	20.5	-	-	-	-
24e124710b423527	1749946183884	60.3	16.4	765	-	-	-
00137a1000013507	1749947383975	-	-	-	20	100	-
main_meter_1	1749947383214	-	-	-	2000	-	-
pool_chemistry_1	1749947383569	-	-	-	-	-	7.2

The use of sensor identifier and timestamp as a compound key in the DynamoDB table allows me to efficiently query the data based on sensor and time period, as shown in listing 4.10. In this example I use a Docker container hosting a local version of DynamoDB running on my machine for development and testing, without the need to deploy to AWS.